

90-Day SDE-2 / Java Full-Stack Interview Prep

Topic Breakdown — Project vs. Theory vs. DSA

Candidate: Sandip Mondal | Target Roles: SDE-2 / Senior Backend, Java Full Stack Developer | Target Companies: Razorpay, Microsoft, JPMC, Oracle

This document maps the full 90-day preparation syllabus into three tracks: topics implemented hands-on in the Digital Wallet / Payment Ledger project, topics covered as focused theory (concept notes, not naturally implementable inside this project), and DSA problem patterns to be practiced independently on a timed basis.

1. Topics covered inside the Digital Wallet Project

Approach for each topic: implement the feature first without the concept in question, observe and explain the resulting bug/limitation/failure, then introduce the concept to resolve it — problem before solution, every time.

Core Spring Boot

- IoC container & Bean lifecycle (BeanFactory vs ApplicationContext, @PostConstruct, BeanPostProcessor)
- Bean scopes & circular dependency resolution (3-level cache)
- Spring AOP & proxy mechanics (JDK dynamic proxy vs CGLIB)
- Self-invocation issue with @Transactional / @Async
- Auto-configuration internals (@ConditionalOnProperty, custom starter)
- Global exception handling with @ControllerAdvice, custom filters & interceptors

Persistence — PostgreSQL & Hibernate

- Entity lifecycle (transient/managed/detached/removed), first-level cache & dirty checking
- N+1 query problem and fixes (JOIN FETCH, @EntityGraph, batch fetch size)
- Transaction isolation levels & phenomena (dirty/phantom reads)
- Optimistic locking (@Version) vs pessimistic locking (SELECT FOR UPDATE)
- Indexing & query tuning (EXPLAIN ANALYZE, composite index ordering, covering index)
- Table partitioning, HikariCP connection pool tuning
- Scale test: seed millions of transaction rows, diagnose and fix real slow queries

Redis Caching

- Cache-aside / read-through / write-through / write-back strategies
- Eviction policies (LRU, LFU, volatile-ttl)
- Cache avalanche (jittered TTL), cache penetration (Bloom filter), cache stampede (mutex)
- Distributed locking (SETNX + TTL, Redlock, watchdog renewal) for concurrent wallet debits

Kafka Event Streaming

- Topics, partitions, consumer groups, partition-key ordering
- Delivery semantics (at-least-once, exactly-once via Kafka transactions)
- Transactional outbox pattern (solving the DB + Kafka dual-write problem)
- Dead letter queues, retry topics with exponential backoff, consumer lag handling

Fintech-specific Patterns

- Idempotency interceptor (idempotency key, Redis/DB dedup, replay-attack mitigation)
- Saga pattern (choreography vs orchestration) for multi-step transfers, compensating transactions
- Double-entry ledger design (immutable debit/credit rows, balance reconstruction, no drift)
- Webhook delivery with HMAC-SHA256 signature verification and retry backoff

Resilience & Microservices

- Resilience4j: CircuitBreaker (CLOSED/OPEN/HALF_OPEN), RateLimiter, Bulkhead, Fallback

Security

- JWT-based auth (HttpOnly cookies vs LocalStorage risk), Spring Security filter chain
- Keycloak: OAuth2/OpenID Connect identity provider integrated as a Spring Security resource server, instead of hand-rolled JWT issuing — closer to how enterprise/bank auth actually works
- CORS configuration, CSRF tokens, Axios/HTTP interceptors

Testing

- Unit testing with JUnit 5 & Mockito for services, repositories, and controllers
- Testing idempotency and concurrency-sensitive code paths

React Frontend

- Virtual DOM reconciliation, hooks (useEffect pitfalls, useMemo, useCallback)
- State management: Context vs Redux Toolkit/Zustand, client vs server state (React Query)

LLD (embedded in features)

- Strategy pattern (fee/interest calculation), Observer pattern (transaction notifications)
- Thread-safe rate limiter (token bucket) applied to the transfer API

System Design (derived from this project)

- Design the Payment Gateway / Wallet system HLD using the actual architecture built, end to end

DevOps

- Dockerizing the Spring Boot app, PostgreSQL, Redis, and Kafka (docker-compose) for local development
- CI/CD pipelines: GitHub Actions and Jenkins (build, test, containerize, deploy) — Jenkins specifically since it's still common in enterprise/bank environments like JPMC
- Kubernetes: deployment manifests, services, ConfigMaps/Secrets, and running the full stack (app + Postgres + Redis + Kafka) on a local cluster (e.g. Minikube/Kind)

Applied AI

- Fraud/anomaly detection engine: velocity checks, amount-anomaly detection, new-device/location flags, hooked into the transaction pipeline as a pre-commit check
- AI transaction assistant: Spring AI integration, LLM function/tool-calling wired into real services (getBalance, searchTransactions, getFraudFlags) instead of hallucinated answers
- Basic RAG over transaction summaries using pgvector on PostgreSQL
- Prompt design and tool-call safety (constraining what the model is allowed to trigger)

Modern Java in Practice

- Records for DTOs (request/response objects) instead of boilerplate classes

- Sealed interfaces to model business outcomes (e.g. TransactionResult: Success / InsufficientFunds / WalletNotFound / DuplicateRequest) instead of exceptions or nullable returns
- Exhaustive pattern matching for switch over those sealed types, so new outcomes can't be silently unhandled
- Virtual Threads (Project Loom): enabling `spring.threads.virtual.enabled` and benchmarking request throughput vs. platform threads under load, since the wallet API is I/O-heavy (DB, Redis, Kafka per request)

2. Theory-only topics (concept notes, not project-implementable)

These rarely occur naturally inside a normal-scale CRUD/fintech application. Covered as focused notes and, where possible, small standalone code demonstrations — not folded into the main project.

Core Java Concurrency

- Java Memory Model: CPU caches, memory barriers, instruction reordering, volatile visibility
- synchronized bytecode (monitorenter/monitorexit), biased/lightweight/heavy locks
- CAS & lock-free programming (AtomicInteger, AtomicReference, ABA problem)
- ReentrantLock vs synchronized, ReentrantReadWriteLock, StampedLock optimistic reads
- Deadlock / livelock / starvation, thread dump analysis (jstack)
- ThreadPoolExecutor internals, rejection handlers, graceful shutdown
- CountdownLatch vs CyclicBarrier, Semaphore, Exchanger
- CompletableFuture async chaining, ThreadLocal memory leaks in pooled threads, Scoped Values

JVM Internals

- JVM memory architecture (stack frames, heap generations, metaspace)
- GC algorithms: Mark-Sweep-Compact, Minor vs Major GC, G1 GC & ZGC region/colored-pointer internals
- Memory leak diagnosis via Eclipse MAT, OutOfMemoryError analysis
- ClassLoader subsystem & delegation model

Collections Internals

- HashMap bucket indexing, collision resolution, treeification threshold, load factor resizing
- ConcurrentHashMap: segment locks (Java 7) vs CAS + synchronized bins (Java 8+)
- equals()/hashCode() contracts, CopyOnWriteArrayList write-cost trade-offs

Modern Java Language Features (deep internals only)

- Java Stream API internal mechanics: lazy evaluation, short-circuiting, and how custom collectors are built under the hood (the Streams themselves are used throughout the project — only this internals layer is theory)

HLD Fundamentals (conceptual, beyond this project's scale)

- CAP theorem / PACELC, consistent hashing
- Load balancing strategies, CDN basics
- Database sharding strategies at very large scale

3. DSA — practiced independently (timed, isolated drilling)

Not tied to the project. Practiced daily under time pressure (45-60 min per problem), since interviews test solving unseen problems on a clock, which a project cannot simulate.

Pattern	Focus areas
Two pointers & sliding window	Two Sum II, 3Sum, Container With Most Water, Trapping Rain Water, longest substring/subarray va
Binary search	Rotated sorted array search, Koko Eating Bananas, capacity/ship-within-D-days style binary search
Stacks & monotonic stack	Daily Temperatures, Largest Rectangle in Histogram, Min Stack, Evaluate RPN, Asteroid Collision
Linked lists & fast/slow pointers	Reverse (k-group), cycle detection, merge k sorted lists, LRU/LFU cache design
Trees — DFS/BFS & BST	Level order/zigzag traversal, LCA, diameter, max path sum, BST validation/kth smallest, serialize/de
Graphs	Number of islands, clone graph, topological sort (course schedule), union-find, word ladder, bipartite
Shortest path & disjoint sets	Dijkstra, cheapest flights within K stops, Prim's/Kruskal's, Tarjan's critical connections
Heaps & priority queues	Kth largest, top-K frequent, median from data stream, task scheduler, design Twitter
Dynamic programming	Climbing stairs family, house robber, coin change, LIS, Kadane's, 2D DP (unique paths, LCS, edit di
Intervals & backtracking	Merge/insert intervals, meeting rooms II, subsets, permutations, combination sum, N-Queens, Sudok
Tries	Implement Trie, Word Search II, design add-and-search-words structure

4. Project objective & description

What we're building

A digital wallet and payment ledger system, built from scratch in Java 17 / Spring Boot 3.x with PostgreSQL, Redis, and Kafka. Users hold a wallet balance in a given currency and can top up funds or transfer money to other wallets. Every money movement is recorded as an immutable double-entry ledger transaction, made idempotent, auditable, and resilient to concurrent access and partial failures — the same core guarantees a real payment system (like Razorpay) has to provide.

Why this project, specifically

- Fintech-flavored: directly matches the interview topics of Razorpay-style companies (idempotency, saga, double-entry ledgers, webhook delivery) instead of a generic CRUD app
- Naturally forces encounters with real backend problems (race conditions on balance updates, N+1 queries at scale, cache invalidation, dual-write consistency) rather than reading about them
- Produces genuine, quantifiable resume material (query latency improvements, concurrency fixes, throughput numbers) instead of invented metrics
- Extensible enough to layer in applied AI (fraud detection, an LLM-based assistant) without changing the core domain

What it deliberately leaves out

Pure CS-fundamentals that don't naturally arise at this project's scale — JVM/GC internals, low-level concurrency primitives, HashMap/ConcurrentHashMap internals — are not forced into the project. They're covered separately as theory (Section 2), because folding them in artificially would teach the pattern wrong: real systems reach for these tools only when scale or specific constraints demand it.

5. Learning strategy & mentoring approach

Two parallel tracks, not one

- Track A — DSA: solved independently, daily, under time pressure (45-60 min/problem), following the original 90-day timetable's AM/PM slots. This can't be replaced by project work — interviews test solving an unseen problem on a clock, which a project doesn't simulate.
- Track B — Applied project: built together, step by step, mapping each week's Spring/DB/Redis/Kafka/System-Design theme from the timetable onto real, working features in the wallet project.

Problem-first teaching method

For every new concept (Redis, Kafka, locking, AI tool-calling, etc.), the sequence is always: (1) implement the feature without the concept, (2) run into and clearly explain the real issue this causes, (3) then introduce the concept specifically to resolve that issue. Concepts are earned through a failure you've seen, not handed over as "best practice."

How the mentoring works day to day

- Step-by-step guided instruction: you're told which file to create, what code to write, and why — code isn't generated wholesale on your behalf, so the understanding stays with you
- Confirmation gates: schema, design, and approach are confirmed with you before we lock them in and move to the next phase
- Visual, step-by-step explanations (diagrams, ERDs, interactive walkthroughs) wherever a picture clarifies faster than paragraphs
- Theory-only topics (Section 2) are taught as focused concept notes and small standalone demonstrations, kept separate from the main project so they don't get diluted or forced in artificially

Cadence

Weekday nights and weekend project blocks (mirroring the original timetable's rhythm) are used for the applied project; weekday AM/PM slots stay dedicated to DSA. Each project phase closes with a short recap of what problem was solved and why, before moving to the next.